

Web Sanitiser

Final Project Specification

1. Overview

The project consists in the design, implementation, and evaluation of a **web sanitiser** tool written in Rust: a defensive utility that inspects web content (HTML pages, downloaded resources, and embedded assets) and neutralises malicious or unwanted material before it can reach the user's system. The tool is delivered as a **command-line application backed by a reusable Rust library (crate)**, so that its sanitisation engine can be embedded into other system-level pipelines (e.g., download managers, mail clients, local proxies, CI scanners).

The project is intended as a comprehensive exercise in Rust system programming: it requires students to combine safe parsing of untrusted input, concurrent processing of multiple resources, and robust I/O handling, while producing a tool that is measurably useful from a security standpoint.

2. Goals and Motivation

Web content is one of the main vectors through which malicious payloads - cross-site scripting fragments, drive-by download links, suspicious redirects, obfuscated scripts, tracking beacons, and known-malicious URLs - reach end-user systems. A sanitiser sits between raw, untrusted web content and the user, producing a cleaned version of the same content together with a structured report of what was removed and why.

The project pursues three goals:

1. **Design** a modular sanitisation pipeline whose policies (what is considered “malign”) are declarative and configurable rather than hard-coded.
2. **Implement** the pipeline in idiomatic Rust, exploiting the language's guarantees on memory safety and its ecosystem for safe concurrency.
3. **Evaluate** the resulting tool both functionally (does it correctly detect and remove threats?) and non-functionally (throughput, latency, memory footprint, scalability across cores).

3. Functional Requirements

The tool must accept one or more inputs - local HTML/asset files, a directory tree, or a list of URLs to fetch - and produce, for each input, a sanitised output and a machine-readable report (JSON). At minimum, the sanitiser must perform the following classes of checks:

- **HTML structural sanitisation:** parse the HTML into a DOM, then strip or rewrite dangerous constructs such as inline event handlers (onclick, onerror, ...), <script> blocks not matching an allow-list, javascript: and data: URIs, <iframe> and <object> elements pointing to untrusted origins, and meta-refresh redirects.
- **URL and link inspection:** extract every URL from the document (anchors, forms, resource references) and check it against a configurable set of block-lists (malware domains, known trackers, IDN homograph patterns). Suspicious links are either rewritten, replaced with a warning placeholder, or removed depending on policy.
- **Embedded resource handling:** *optionally* fetch referenced sub-resources (CSS, JS, images) and apply per-type sanitisation, with a strict bound on depth, total bytes, and number of requests.
- **Reporting:** for every input, emit a structured report listing each sanitisation action (rule fired, location in the document, original fragment, replacement) so the tool is auditable and the evaluation phase is reproducible.

The CLI must support batch mode (a list of inputs), configurable policies via a configuration file, selectable output verbosity, and a non-zero exit code when the sanitiser would refuse content outright.

The rules above should defend against attacks such as:

- [XSS](#): prevent client-side script injection
- [MIME confusion](#): do not trust the mime type or content type declared by a resource, but validate via content sniffing
- DoS attacks: prevent various types of DoS attacks, like [compression bombs](#), [XML bombs](#) or huge images, which cause resource exhaustion, or timeout attacks like [Slowloris](#).
- Active document content: check for active content in documents, like javascript inside of a [PDF document](#).
- Unicode normalization confusion in URLs, that can cause vulnerabilities such as [Host/Split](#)
- [SSRF](#) (based on the optional sub-resource fetching): prevent a server into making unintended requests to internal or external systems on the attacker's behalf

4. Non-Functional Requirements

The sanitiser must process untrusted input safely: no parser panic, no unbounded memory allocation, and no path-traversal or SSRF when resolving sub-resources. It must complete within configurable per-input time and size budgets and degrade gracefully when those budgets are exceeded. Throughput should scale with the number of available cores when processing batches.

5. System-Programming Aspects in Rust

The project is the place where students must demonstrate command of the system-level features of Rust. Two areas are mandatory.

5.1 Concurrency

Batch processing must be concurrent. Students are expected to design a pipeline where a pool of worker threads consumes inputs from a shared queue, with results and reports flowing back through channels. Synchronisation of shared state - caches of resolved URLs, statistics, block-list lookups - must be implemented with appropriate primitives (*Arc*, *Mutex/RwLock*, atomics) and the rationale for each choice must be discussed in the report.

5.2 Memory Safety and Zero-Copy Parsing

The sanitiser handles attacker-controlled bytes; a memory bug here would defeat the tool's purpose. Students must rely on safe Rust throughout the parsing layer and adopt zero-copy techniques wherever practical like borrowing, copy-on-write semantics and streaming tokenizers/rewriters (e.g., *lol_html/html5ever/tendril*) that expose the document without materialising redundant copies; justify your ownership model in the report. Use of *unsafe* is discouraged; if introduced, every *unsafe* block must be justified, encapsulated, and accompanied by a soundness argument.

The interaction between these two axes - sharing borrowed data across threads - is itself an interesting design problem and should be addressed explicitly (lifetimes vs. *Arc*<[u8]>-backed buffers, sendability of parser state, etc.).

6. Suggested Architecture

A reasonable decomposition, which students may revise:

- **Input layer** that abstracts over local files and remote fetches.
- **Parsing layer** built on top of an existing safe HTML parser.
- **Core engine** that applies a sequence of sanitisation rules dictated by a configurable policy layer, with a scheduler distributing inputs across workers and aggregating reports.
- **CLI front-end** that exposes the library and produces human- and machine-readable output.

The split between binary crate (CLI) and library crate (engine) must be clean, so that the engine is reusable and independently testable.

7. Evaluation

The project is not complete without an experimental evaluation. Students must build a corpus of test inputs containing both benign pages and crafted malicious samples (synthetic XSS payloads, obfuscated

scripts, links to entries on public block-lists, oversized inputs, malformed HTML). On this corpus they must measure:

- **Correctness:** detection rate on malicious samples and false-positive rate on benign pages, against a documented ground truth.
- **Performance:** throughput (inputs/second) and per-input latency as a function of input size, with and without sub-resource fetching.
- **Scalability:** speed-up curves as the number of worker threads grows, with a discussion of where the bottleneck lies (parser, I/O, contention on shared state).
- **Resource usage:** peak memory and how it relates to the chosen zero-copy strategies.

Results must be presented with plots and a critical discussion.

8. Deliverables

A Git repository containing the source code, a build that runs with stable Rust, unit and integration tests with reasonable coverage of the sanitisation rules, the test corpus and benchmark scripts used for the evaluation, and a final report (≈15-25 pages) covering: requirements analysis, architectural decisions, a focused discussion of the Rust system-programming aspects (concurrency model, lifetimes, error handling, unsafe if any), the experimental evaluation, and a critical assessment of limitations and possible extensions.

9. Out of Scope

The tool is not an antivirus, does not perform dynamic analysis or sandboxed execution of scripts, and is not expected to defeat advanced obfuscation. Its goal is principled, auditable sanitisation according to declared policies - not heuristic threat detection.